

Name: Krish Agrawal

Roll No: 381041

Batch: A2

Assignment No: 1

Problem Statement:

Implement a basic probabilistic generative model such as Gaussian Mixture Models. Develop a simple text generation system using Markov Chains.

Objective:

- To explore probabilistic generative models and understand how they capture training data distributions.
- To study the Markov property and its application to word transition modelling in text sequences.
- To implement a first-order Markov Chain text generator with `train()` and `generate()` methods.
- To construct a reusable `MarkovTextGenerator` class and validate it across multiple starting words.
- To examine the coherence of generated outputs and understand the limitations of memoryless generation.

S/W Packages and H/W apparatus used:

- Operating System: Windows/Linux/macOS
- Kernel: Python 3.x
- Tools: Jupyter Notebook / Anaconda
- Hardware: CPU with minimum 4GB RAM
- Libraries and packages used: Python `random` module, Python `collections` (`defaultdict`)

Theory:

Generative models are a category of machine learning systems that learn the probability distribution of training data and can sample new instances from that distribution. In contrast to discriminative models that focus on class boundaries, generative models capture the underlying data generation process. Notable examples include Gaussian Mixture Models (GMMs), Hidden Markov Models, Variational Autoencoders, and Generative Adversarial Networks.

Markov Chains:

A Markov Chain is a stochastic sequence in which the probability of moving to the next state depends solely on the current state — not on any earlier history. This memoryless characteristic is called the Markov property. In text generation, each word is treated as a state, and the likelihood of the following word is learned from corpus statistics. A first-order chain uses a single preceding word as context, while

higher-order variants consider longer histories.

Model Training:

Training involves scanning consecutive word pairs (w_i, w_{i+1}) in the lowercased corpus and recording each successor in a defaultdict of lists. Because frequent successors appear multiple times in

the list, random sampling implicitly reflects Maximum Likelihood Estimation of transition probabilities without explicit normalisation.

Text Generation:

Starting from a seed word, the generator samples the next word from the model's successor list using `random.choice()`. When a word has no recorded successors, a random vocabulary word is chosen as fallback. This loop continues until the desired output length is reached, and all tokens are joined into a readable string.

Gaussian Mixture Models (GMM):

A GMM represents the data distribution as a weighted sum of Gaussian components, each with its own mean and covariance. Training uses the Expectation-Maximization algorithm, alternating between soft assignment of points to components (E-step) and parameter updates (M-step). While Markov Chains handle discrete sequential data, GMMs are suited for continuous feature distributions and are widely applied to clustering and density estimation tasks.

Methodology:

- Import Python's random module and defaultdict from the collections library.
- Define the MarkovTextGenerator class with an `__init__` method initializing `self.model` as a defaultdict of lists.
- Implement `train(text)`: lowercase and split the corpus, then for every consecutive word pair append the successor to the predecessor's list.
- Implement `generate(start_word, length)`: seed the output list with the start word and iteratively sample successors; fall back to a random vocabulary word at dead ends.
- Construct a six-sentence training corpus spanning deep learning, machine learning, data science, and AI.
- Instantiate the generator, call `train()`, then generate sequences of length 12 starting from 'learning', 'data', and 'models'.

Use Cases:

- Use Case 1 - Content Drafting Assistance: Domain-specific Markov models trained on technical documents can generate rough draft text for reports or product descriptions, giving writers a faster starting point.
- Use Case 2 - Training Data Augmentation: Synthetic sentences generated by the Markov model can supplement small NLP datasets, enriching classifiers with statistically consistent additional examples.
- Use Case 3 - Rapid Chatbot Prototyping: For narrowly scoped domains, Markov generators offer a lightweight alternative to neural chatbots when speed of deployment matters more than response diversity.

Advantages:

- Requires no external libraries — only Python built-ins are needed.
- Training completes in a single corpus pass, making it near-instant even for large texts.
- The transition table is fully transparent and easy to inspect.
- Extending to bigram or trigram context needs minimal code changes.
- Stochastic sampling naturally varies outputs across runs without any configuration.

Limitations:

- One-word memory prevents coherence over longer text sequences.
- Out-of-vocabulary words trigger random fallbacks, breaking thematic continuity.
- No sentence boundary or punctuation awareness causes run-on outputs.
- Highly repetitive results emerge from small training corpora.
- Global topic consistency cannot be maintained without longer context windows.

Applications:

- Next-word suggestion in lightweight keyboard autocomplete systems.
- Randomized story-starter and plot-prompt generation for creative writing tools.
- Synthetic spam corpus creation for training email spam detectors.
- Stylistically consistent melody or lyric generation by treating notes as states.
- NPC dialogue variation in video games trained on scripted conversation data.

Conclusion:

This assignment developed a first-order Markov Chain text generator using Python's defaultdict. A MarkovTextGenerator class with train() and generate() methods was constructed and trained on a six-sentence AI and data science corpus. Text sequences seeded from 'learning', 'data', and 'models' were successfully generated, reflecting learned word transition statistics. The locally plausible but globally incoherent outputs highlight the core limitation of the memoryless Markov assumption, motivating the progression towards richer generative architectures such as GMMs, HMMs, and modern deep neural language models.